

LAB-01 ASSIGNMENT

Building a Spring Boot Books REST API

Enterprise Web Application Development | Kabul University

Course: Enterprise Web Application Development

Lab Topic: Spring Boot REST API Fundamentals

Date: 9 September 2026

Student Name	Abdul_Rahman_Ayoubi
Student ID	S01020102
Section / Class	IS

1. Lab Objectives

- Create and configure a Spring Boot web project.
- Create the Books.java model class.
- Create a REST controller for /api/v1/books.
- Implement HTTP GET to return all books as JSON.
- Run and test the API and verify the HTTP response.

2. Project Configuration

Setting	Value
Project	Maven
Language	Java
Group	edu.ku
Artifact	book-api
Name	book-api
Package Name	edu.ku.bookapi
Packaging	Jar
Java	16
Dependency	Spring Web

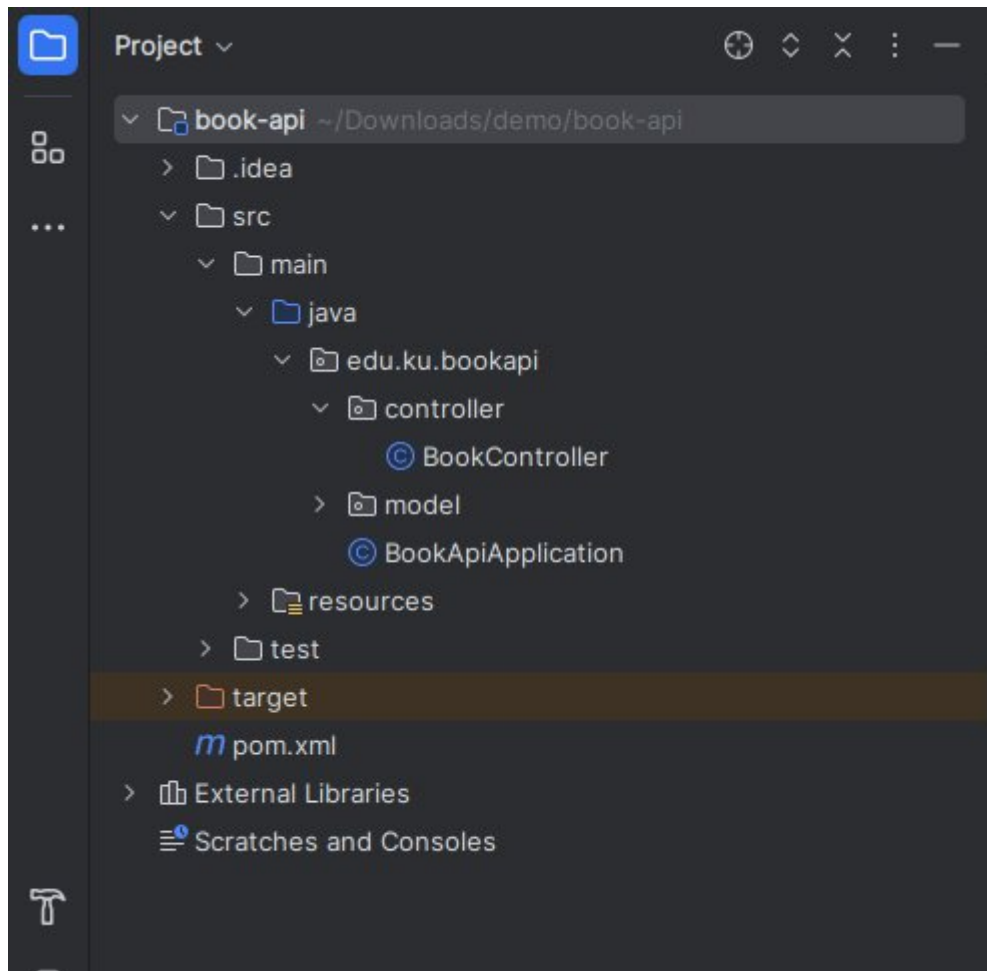
3. Application Configuration

src/main/resources/application.properties

spring.application.name=book-api

server.port=8080

4. Project Structure



5. Application Startup

```
Run BookApiApplication x
/home/rahman/.jdk/ms-21.0.12.1/bin/java ...

:: Spring Boot ::      (v3.5.6)

2026-09-18T19:24:17.488+04:30 INFO 45259 --- [book-api] [main] edu.ku.bookapi.BookApiApplication : Starting BookApiApplication using Ja
2026-09-18T19:24:17.501+04:30 INFO 45259 --- [book-api] [main] edu.ku.bookapi.BookApiApplication : No active profile set, falling back
2026-09-18T19:24:20.209+04:30 INFO 45259 --- [book-api] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (h
2026-09-18T19:24:20.247+04:30 INFO 45259 --- [book-api] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-18T19:24:20.248+04:30 INFO 45259 --- [book-api] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tom
2026-09-18T19:24:20.432+04:30 INFO 45259 --- [book-api] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebAppl
2026-09-18T19:24:20.439+04:30 INFO 45259 --- [book-api] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initiali
2026-09-18T19:24:21.954+04:30 INFO 45259 --- [book-api] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http w
2026-09-18T19:24:21.974+04:30 INFO 45259 --- [book-api] [main] edu.ku.bookapi.BookApiApplication : Started BookApiApplication in 6.451
```

6. API Testing

```
Pretty-print ☒

{"id": 1,
 "title": "Clean Code",
 "author": "Robert C. Martin",
 "isbn": "9780132350884",
 "publishedYear": 2008,
 "category": "Software Engineering"
},
{
  "id": 2,
  "title": "Effective Java",
  "author": "Joshua Bloch",
  "isbn": "9780134685991",
  "publishedYear": 2018,
  "category": "Java"
},
{
  "id": 3,
  "title": "Designing Data-Intensive Applications",
  "author": "Martin Kleppmann",
  "isbn": "9781449373320",
  "publishedYear": 2017,
  "category": "Distributed Systems"
},
{
  "id": 4,
  "title": "Spring in Action",
  "author": "Craig Walls",
  "isbn": "9781617297571",
  "publishedYear": 2022,
  "category": "Spring"
},
{
  "id": 5,
  "title": "Computer Networks",
  "author": "Andrew S. Tanenbaum",
  "isbn": "9780132126953",
  "publishedYear": 2010,
  "category": "Networking"
},
{
  "id": 6,
  "title": "The Pragmatic Programmer",
  "author": "Andrew Hunt",
  "isbn": "9780135957059",
  "publishedYear": 2019,
  "category": "Software Engineering"
}
```

7. Get book by ID:

```
Pretty-print ☒
```

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "isbn": "9780132350884",
  "publishedYear": 2008,
  "category": "Software Engineering"
}
```

8. Create book in postman:

The screenshot shows the Postman application interface. On the left, the 'Collections' panel shows 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The main panel displays a POST request to 'http://localhost:8080/api/v1/books'. The request body is a JSON object with the following fields: 'title' (The Pragmatic Programmer), 'author' (Andrew Hunt), 'isbn' (9780135957059), 'publishedYear' (2019), and 'category' (Software Engineering). The response is also a JSON object with the same fields, but the 'id' is 7. The status bar at the bottom indicates '201 Created' with a response time of 10 ms and a size of 313 B.

POST Get data

My Collection / Get data

POST http://localhost:8080/api/v1/books

Body

```
1 {
2   "title": "The Pragmatic Programmer",
3   "author": "Andrew Hunt",
4   "isbn": "9780135957059",
5   "publishedYear": 2019,
6   "category": "Software Engineering"
7 }
```

Body Cookies Headers (5) Test Results (0/1)

201 Created · 10 ms · 313 B

Save Response

JSON Preview Visualize

```
1 {
2   "id": 7,
3   "title": "The Pragmatic Programmer",
4   "author": "Andrew Hunt",
5   "isbn": "9780135957059",
6   "publishedYear": 2019,
7   "category": "Software Engineering"
8 }
```

9. Reflection Questions and Answers

1. Why is GET appropriate for /api/v1/books?

GET is appropriate because the endpoint retrieves/reads the collection of books. It does not create, modify, or delete data, and no request body is required.

2. What is the purpose of @RestController?

@RestController tells Spring that the class handles REST requests. Its return values are written to the HTTP response body and are normally serialized as JSON.

3. What does <Books> mean in List<Books>?

<Books> is the generic type parameter. It means the list can contain Books objects, so Java knows the type of each element.

4. Why is books plural in the URI, and what does v1 represent?

"books" is plural because the endpoint represents a collection of book resources. "v1" represents version 1 of the API.

5. How does a Java List<Books> become JSON without manually writing JSON code?

The controller returns the Java List<Books>. Spring Web uses message conversion with Jackson to serialize the Java objects and list into a JSON array.

6. What would happen if BookController were placed outside the package hierarchy scanned by the Spring Boot application?

Spring Boot might not discover the controller during component scanning. The endpoint would then not be registered and /api/v1/books would typically return 404 Not Found.

10. Completion Checklist

- ☒ Project opens and builds successfully.
- ☒ Spring Boot starts on port 8080.
- ☒ Books.java exists under the model package.
- ☒ BookController.java exists under the controller package.
- ☒ GET /api/v1/books returns all book objects.
- ☒ GET /api/v1/books/{id} returns a book by its ID.
- ☒ POST /api/v1/books adds a new book.
- ☒ GET /api/v1/books returns HTTP 200 OK.
- ☒ GET /api/v1/books/{id} returns HTTP 200 OK when the ID exists.
- ☒ GET /api/v1/books/{id} returns HTTP 404 Not Found when the ID does not exist.
- ☒ POST /api/v1/books returns HTTP 201 Created when a book is added successfully.
- ☒ Response content is JSON.
- ☒ Required screenshots are inserted.
- ☒ Reflection questions are completed.
- ☒ The project uses /api/v1/books for all API endpoints.
- ☒ The final submission is ready.